the city, and it usually would take slightly short of 3 hours. However, your navigator realizes that in the next 10 miles on that route there is heavy traffic, while going around Rome on the West side, the traffic is all clear: if the navigator just used Dijkstra, the next edge to expand would be the shorter time for the next 10 miles, and it would lead you to go West.

Unfortunately, that would add at least 1 hour to your trip: A* can come to the rescue, and balance the short-term advantage of a traffic-free motorway section with the long-term gain of a shorter and usually faster route.

Although this example is extremely simplified, you get the point: A* can better balance long-term costs with local choices, and that's why it had been the state-of-the-art in AI, for pathfinding, for instance in video games[15].

The next step to improve these navigation algorithms would be considering how requests for shortest path are handled "in a vacuum": especially during rush hours, when many users will request the shortest path between similar locations, suggesting the same route to all of them could led to unnecessarily high traffic on some roads, and very low traffic on others. Wouldn't it be great if a more balanced routing system could take into consideration all requests involving the same segments, spread the traffic over several routes, and minimize the congestion due to vehicles sharing the same road?

That goal is ambitious, out of scope for this book, and out of reach with classic computers: that's why quantum developers[16] are working on it, with encouraging results.